USER'S GUIDE TO TF_MISFIT_GOF_CRITERIA

Julia Version

Computation of the Time-Frequency Misfit and Goodness-of-Fit Criteria for Comparing Time Signals

Adapted from the original Fortran95 guide by
Miriam Kristeková, Jozef Kristek, and Peter Moczo
for the Julia port

Martí Circuns-Duxans
Barcelona Supercomputing Center
marti.circuns@bsc.es
https://github.com/MartiCD

March 27, 2026

1 Introduction

1.1 Purpose

The program `TF_MISFIT_GOF_CRITERIA` is designed for the computation of time-frequency misfit and time-frequency goodness-of-fit (GOF) criteria for comparing two time signals. The Julia version reproduces the workflow of the original Fortran95 implementation, including:

- continuous wavelet transform (CWT) based on the Morlet wavelet,
- time-frequency envelope misfit and phase misfit,
- time-dependent and frequency-dependent marginal misfits,
- scalar envelope and phase misfit measures,
- corresponding goodness-of-fit criteria.

1.2 Origin of the method

This Julia guide is adapted from the original user guide for the Fortran95 code. The original guide states that the program is intended for the computation of time-frequency misfit and goodness-of-fit criteria for comparing time signals, and documents the input files, output files, and overall workflow of the code.

1.3 Recommended reference

Users of the Julia version should cite the method papers:

- Kristekova, M., Kristek, J., Moczo, P., Day, S. M., 2006. *Misfit Criteria for Quantitative Comparison of Seismograms.* Bull. Seism. Soc. Am. 96(5), 1836–1850.

- Kristeková, M., Kristek, J., Moczo, P., 2009. *Time-frequency misfit and goodness-of-fit criteria for quantitative comparison of time signals.* Geophys. J. Int. 178, 813–825.

2 Program Overview

The Julia implementation follows the same structure as the original code:

1. read control parameters from the auxiliary input file,
2. read two input time signals,
3. compute the Fourier-domain representation of the signals,
4. compute the continuous wavelet transform,
5. evaluate envelope and phase misfit measures,
6. write scalar, time-dependent, frequency-dependent, and time-frequency outputs to ASCII files.

The program supports both:

- **global normalization**, and
- **local normalization**.

3 Running the Julia Program

Assuming the Julia source file is named `tf_misfit_port.jl`, run:

```
julia tf_misfit_port.jl HF_TF-MISFIT_GOF
```

If no file is provided, the code may default to:

```
HF_TF-MISFIT_GOF
```

4 Input Files

The Julia version requires the same three logical inputs as the original program:

- an auxiliary file containing input parameters,
- a file containing the first signal,
- a file containing the second signal (or reference signal).

4.1 Auxiliary file `HF_TF-MISFIT_GOF`

The file type is ASCII and contains several control parameters.

```
&INPUT
  S1_NAME = 'signal1.dat',
  S2_NAME = 'signal2.dat',
  NC = 1,
  MT = 3000,
  DT = 0.01,
  FMIN = 0.1,
```

```
  FMAX = 10.0,
  IS_S2_REFERENCE = .FALSE.,
  LOCAL_NORM = .FALSE.
/
```

4.2 Meaning of variables

Variable	Type	Description
`S1_NAME`	string	Name of the file containing the first signal.
`S2_NAME`	string	Name of the file containing the second signal.
`NC`	integer	Number of components of each signal. Default: `NC = 1`.
`MT`	integer	Number of time samples in each signal.
`DT`	real	Time step in seconds.
`FMIN, FMAX`	real	Minimum and maximum frequency in Hz defining the desired frequency range for the misfit criteria. The frequency range is sampled logarithmically.
`IS_S2_REFERENCE`	logical	`.TRUE.`: signal S2 is taken as reference. `.FALSE.`: neither signal is forced to be the reference. Default: `.FALSE.`.
`LOCAL_NORM`	logical	`.TRUE.`: local normalization is used. `.FALSE.`: global normalization is used. Default: `.FALSE.`.

4.3 Input signal files

Each signal file is an ASCII file. The expected format is one row per time sample:

```
time s1_component1 s1_component2 ... s1_componentNC
```

Likewise for the second signal. In the original formulation, the data are read as:

```
DO I = 1, MT
  READ(...,*) TIME, ( S1(J,I), J = 1, NC )
END DO
```

Therefore, the first column is the time value and the remaining `NC` columns are the components of the signal at that time sample.

5 Output Files

The Julia version produces the same family of ASCII output files as the original code.

5.1 Summary file `MISFIT-GOF.DAT`

This file contains:

1. frequency bounds,
2. time-frequency grid dimensions,
3. time step and number of components,
4. maximum signal amplitude,

5. scalar envelope and phase misfits,
6. scalar goodness-of-fit values,
7. maxima of the main time-frequency and marginal outputs.

Its logical structure is:

```
FMIN FMAX
NF_TF MT
DT NC
MAX(ABS(S1), ABS(S2))
EM(1) PM(1)
...
EM(NC) PM(NC)
EG(1) PG(1)
...
EG(NC) PG(NC)
MAXVAL(ABS(TFEM)) MAXVAL(ABS(TFPM))
MAXVAL(ABS(FEM)) MAXVAL(ABS(FPM))
MAXVAL(ABS(TEM)) MAXVAL(ABS(TPM))
MAXVAL(ABS(TFRS1)) MAXVAL(ABS(TFRS2))
```

5.2 List of generated output files

For component index x, the code may generate:

File	Contents
`S1.DAT, S2.DAT`	Input signals as read by the program.
`MISFIT-GOF.DAT`	Control data and scalar misfit / GOF measures.
`TFEMx.DAT`	Time-frequency envelope misfit.
`TFEGx.DAT`	Time-frequency envelope goodness-of-fit.
`TFPMx.DAT`	Time-frequency phase misfit.
`TFPGx.DAT`	Time-frequency phase goodness-of-fit.
`TEMx.DAT`	Time-dependent envelope misfit.
`TEGx.DAT`	Time-dependent envelope goodness-of-fit.
`TPMx.DAT`	Time-dependent phase misfit.
`TPGx.DAT`	Time-dependent phase goodness-of-fit.
`FEMx.DAT`	Frequency-dependent envelope misfit.
`FEGx.DAT`	Frequency-dependent envelope goodness-of-fit.
`FPMx.DAT`	Frequency-dependent phase misfit.
`FPGx.DAT`	Frequency-dependent phase goodness-of-fit.
`TFRS1_x.DAT`	Time-frequency representation of the first signal.
`TFRS2_x.DAT`	Time-frequency representation of the second signal.

5.3 Shapes and write patterns

The output layout follows the original logic.

Time-frequency files For files such as `TFEMx.DAT`, `TFEGx.DAT`, `TFPMx.DAT`, `TFPGx.DAT`, `TFRS1_x.DAT`, and `TFRS2_x.DAT`, the code writes:

```
DO L = 1, NF_TF
  WRITE(...) ( ARRAY(x,I,L), I = 1, MT )
END DO
```

Hence each logical frequency row contains MT values.

Time-dependent files For `TEMx.DAT`, `TEGx.DAT`, `TPMx.DAT`, `TPGx.DAT`:

```
DO I = 1, MT
  WRITE(...) ARRAY(x,I)
END DO
```

Frequency-dependent files For `FEMx.DAT`, `FEGx.DAT`, `FPMx.DAT`, `FPGx.DAT`:

```
DO L = 1, NF_TF
  WRITE(...) ARRAY(x,L)
END DO
```

6 Internal Computation

6.1 Wavelet transform

The code computes a continuous wavelet transform using a Morlet wavelet in the frequency domain. The central nondimensional frequency parameter is typically `W0 = 6.0`, following the original implementation.

6.2 Misfit quantities

The Julia code computes:

- **TFEM**: time-frequency envelope misfit,
- **TFPM**: time-frequency phase misfit,
- **TEM**: time-dependent envelope misfit,
- **TPM**: time-dependent phase misfit,
- **FEM**: frequency-dependent envelope misfit,
- **FPM**: frequency-dependent phase misfit,
- **EM**: scalar envelope misfit,
- **PM**: scalar phase misfit.

The corresponding goodness-of-fit criteria are computed from the misfits and written to:

- `TFEGx.DAT`, `TFPGx.DAT`,
- `TEGx.DAT`, `TPGx.DAT`,
- `FEGx.DAT`, `FPGx.DAT`,
- the scalar GOF values stored in `MISFIT-GOF.DAT`.

7 Practical Notes

- The Julia port preserves the file naming and output organization of the original Fortran program, making it compatible with the legacy MATLAB plotting scripts after minor adjustments.
- Since the summary file `MISFIT-GOF.DAT` is not a rectangular table, post-processing scripts should read it line by line rather than with generic matrix readers.
- Time-frequency output files are logically $NF_TF \times MT$, even when long rows may wrap visually in text editors.

8 Example Workflow

1. Prepare `HF_TF-MISFIT_GOF`.
2. Prepare the two signal files.
3. Run:

```
julia tf_misfit_port.jl HF_TF-MISFIT_GOF
```

4. Inspect:

```
MISFIT-GOF.DAT
TFEM1.DAT
TFEG1.DAT
TFPM1.DAT
TFPG1.DAT
...
```

5. Plot the results with Julia, Python, or adapted MATLAB scripts.

9 Acknowledgement

The original methodology and user guide are due to Miriam Kristeková, Jozef Kristek, and Peter Moczo. This document is a Julia-oriented adaptation of the original Fortran95 user guide.